Introduction to Vertex AI Pipelines

To learn more, run the "Vertex AI Pipelines: Lightweight Python function-based components, and component I/O" notebook in one of the following environments:

Open in Colab (https://colab.research.google.com/github/GoogleCloudPlatform/vertex-ai-samples/blob/main/notebooks/official/pipelines/lightweight_functions_component_io_kfp.ipynb)

|

Open in Colab Enterprise (https://console.cloud.google.com/vertex-ai/colab/import/https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fvertex-ai-samples%2Fmain%2Fnotebooks%2Fofficial%2Fpipelines%2Flightweight_functions_component_io_kfp.ipynb)

|

Open in Vertex AI Workbench (https://console.cloud.google.com/vertex-ai/workbench/deploy-notebook?download_url=https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fvertex-ai-samples%2Fmain%2Fnotebooks%2Fofficial%2Fpipelines%2Flightweight_functions_component_io_kfp.ipynb)

|

View on GitHub (https://github.com/GoogleCloudPlatform/vertex-ai-samples/blob/main/notebooks/official/pipelines/lightweight_functions_component_io_kfp.ipynb)

Vertex AI Pipelines lets you automate, monitor, and govern your machine learning (ML) systems in a serverless manner by using ML pipelines to orchestrate your ML workflows. You can batch run ML pipelines defined using the Kubeflow Pipelines or the TensorFlow Extended (TFX) framework. To learn how to choose a framework for defining your ML pipeline, see Interfaces to define a pipeline (/vertex-ai/docs/pipelines/interfaces#define).

This page provides an overview of the following:

- What is an ML pipeline? (#ml-pipeline)
- Structure of an ML pipeline (#structure)
- Pipeline tasks and components (#tasksandcomponents)
- Life cycle of an ML pipeline (#life-cycle)
- Use Vertex ML Metadata to track the lineage of ML artifacts (#track-lineage)
- Add pipeline runs to experiments (#experiments)

Note: If you're experienced in creating ML pipelines using the Kubeflow Pipelines SDK and want to understand the differences between Vertex AI Pipelines and Kubeflow Pipelines, see Migrate from Kubeflow Pipelines to Vertex AI Pipelines (/vertex-ai/docs/pipelines/migrate-kfp).

What is an ML pipeline?

An ML pipeline is a portable and extensible description of an MLOps workflow as a series of steps called pipeline tasks. Each task performs a specific step in the workflow to train and deploy an ML model.

With ML pipelines, you can apply MLOps strategies to automate and monitor repeatable processes in your ML practice. For example, you can reuse a pipeline definition to continuously retrain a model on the latest production data. For more information about MLOps in Vertex AI, see MLOps on Vertex AI (/vertex-ai/docs/start/introduction-mlops).

Structure of an ML pipeline

An ML pipeline is a directed acyclic graph (DAG) of containerized pipeline tasks that are interconnected using input-output dependencies. You can author each task either in Python or as a prebuilt container images.

You can define the pipeline as a DAG using either the Kubeflow Pipelines SDK or the TFX SDK, compile it to its YAML for intermediate representation, and then run the pipeline. By default,

pipeline tasks run in parallel. You can link the tasks to execute them in series. For more information about pipeline tasks, see Pipeline task (#pipeline-task). For more information about the workflow for defining, compiling, and running the pipeline, see Life cycle of an ML pipeline (#life-cycle).

Click here for an example illustrating input-output dependencies between tasks within an ML pipeline.

Consider an ML pipeline with the following steps:

- **Prepare data**: Prepare or preprocess training data.
  - Input (from tasks within the same ML pipeline): None.
  - Output: Prepared or preprocessed training data.
- **Train model**: Use the prepared training data to train a model.
  - Input: Prepared or preprocessed training data from pipeline task **Prepare data**.
  - Output: Trained model.
- **Evaluate model**: Evaluate the trained model.

  Input: Trained model from pipeline task **Train model**.
- **Deploy**: Deploy the trained model for predictions.

  Input: Trained model from pipeline task **Train model**.

When you compile your ML pipeline, the pipelines SDK you're using (Kubeflow Pipelines or TFX) analyzes the data dependencies between these tasks and creates the following workflow DAG:

- **Prepare data** doesn't rely on other tasks within the same ML pipeline for inputs. Therefore, it can be the first step in the ML pipeline, or run concurrently with other tasks.
- **Train model** relies on **Prepare data** for inputs. Therefore, it occurs after **Prepare data**.
- **Evaluate** and **Deploy** both depend on the trained model. Therefore, they can run concurrently, but after **Train model**.

When you run your ML pipeline, Vertex AI Pipelines executes these tasks in the sequence described in the DAG.

Pipeline tasks and components

A pipeline task (#pipeline-task) is an instantiation of a pipeline component (#pipeline-component) with specific inputs. While defining your ML pipeline, you can interconnect multiple tasks to form a DAG, by routing the outputs of one pipeline task to the inputs for the next pipeline task in the ML workflow. You can also use the inputs for the ML pipeline as the inputs for a pipeline task.

Pipeline component

A pipeline component is a self-contained set of code that performs a specific step of an ML workflow, such as data preprocessing, model training, or model deployment. A component typically consists of the following:

- **Inputs**: A component might have one or more input parameters and artifacts.
- **Outputs**: Every component has one or more output parameters or artifacts.
- **Logic**: This is the component's executable code. For containerized components, the logic also contains the definition of the environment, or container image, where the component runs.

Components are the basis of defining tasks in an ML pipeline. To define pipeline tasks, you can either use predefined Google Cloud Pipeline Components (/vertex-ai/docs/pipelines/components-introduction) or create your own custom components.

Predefined components

Use predefined Google Cloud Pipeline Components if you want to use features of Vertex AI, such as AutoML, in your pipeline. To learn how to use Google Cloud Pipeline Components to define a pipeline, see Build a Pipeline (/vertex-ai/docs/pipelines/build-pipeline).

Custom components

You can author your own custom components to use in your ML pipeline. For more information about authoring custom components, see Build your own pipeline components (/vertex-ai/docs/pipelines/build-own-components).

To learn how to author custom Kubeflow Pipelines components, see the "Pipelines with lightweight components based on Python functions" (https://github.com/GoogleCloudPlatform/vertex-ai-samples/blob/main/notebooks/official/pipelines/lightweight_functions_component_io_kfp.ipynb) tutorial notebook on GitHub. To learn how to author custom TFX components, see the TFX Python function component tutorial (https://www.tensorflow.org/tfx/tutorials/tfx/python_function_component) on the TensorFlow Extended in Production tutorials (https://www.tensorflow.org/tfx/tutorials#tensorflow-in-production-tutorials).

Pipeline task

A pipeline task is the instantiation of a pipeline component and performs a specific step in your ML workflow. You can author ML pipeline tasks either using Python or as prebuilt container images.

Within a task, you can build on the on-demand compute capabilities of Vertex AI with Kubernetes to scalably execute your code, or delegate your workload to another execution engine, such as BigQuery, Dataflow, or Google Cloud Serverless for Apache Spark.

Lifecycle of an ML pipeline

From definition to execution and monitoring, the lifecycle of an ML pipeline comprises the following high-level stages:

1. **Define**: The process of defining an ML pipeline and its task is also called building a pipeline. In this stage, you need to perform the following steps:
    a. **Choose an ML framework**: Vertex AI Pipelines supports ML pipelines defined using the TFX or Kubeflow Pipelines framework. To learn how to choose a framework for building your pipeline, see Interfaces to define a pipeline (/vertex-ai/docs/pipelines/interfaces#define).
    b. **Define pipeline tasks and configure pipeline**: For more information, see Build a Pipeline (/vertex-ai/docs/pipelines/build-pipeline).
2. **Compile**: In this stage, you need to perform the following steps:
    a. Generate your ML pipeline definition in a compiled YAML file for intermediate representation, which you can use to run your ML pipeline.

   b. Optional: You can upload the compiled YAML file as a pipeline template to a repository and reuse it to create ML pipeline runs.

3. **Run**: Create an execution instance of your ML pipeline using the compiled YAML file or a pipeline template. The execution instance of a pipeline definition is called a pipeline run (#pipeline-run-definition).

   You can create a one-time occurrence of a pipeline run or use the scheduler API (/vertex-ai/docs/pipelines/schedule-pipeline-run) to create recurring pipeline runs from the same ML pipeline definition. You can also clone an existing pipeline run. To learn how to choose an interface to run an ML pipeline, see Interfaces to run a pipeline (/vertex-ai/docs/pipelines/interfaces#run). For more information about how to create a pipeline run, see Run a pipeline (/vertex-ai/docs/pipelines/run-pipeline).

4. **Monitor, visualize, and analyze runs**: After you create a pipeline run, you can do the following to monitor the performance, status, and costs of pipeline runs:

   - Configure email notifications for pipeline failures. For more information, see Configure email notifications (/vertex-ai/docs/pipelines/email-notifications).
   - Use Cloud Logging to create log entries for monitoring events. For more information, see View pipeline job logs (/vertex-ai/docs/pipelines/logging).
   - Visualize, analyze, and compare pipeline runs. For more information, see Visualize and analyze pipeline results (/vertex-ai/docs/pipelines/visualize-pipeline).
   - Use Cloud Billing export to BigQuery to analyze pipeline run costs. For more information, see Understand pipeline run costs (/vertex-ai/docs/pipelines/understand-pipeline-cost-labels).

5. **Optional: stop or delete pipeline runs**: There is no restriction on how long you can keep a pipeline run active. You can optionally do the following:

   - Stop a pipeline run.
   - Pause or resume a pipeline run schedule.
   - Delete an existing pipeline template, pipeline run, or pipeline run schedule.

What is a pipeline run?

A pipeline run is an execution instance of your ML pipeline definition. Each pipeline run is identified by a unique run name. Using Vertex AI Pipelines, you can create an ML pipeline run in the following ways:

- Use the compiled YAML definition of a pipeline
- Use a pipeline template from the Template Gallery

For more information about how to create a pipeline run, see Run a pipeline (/vertex-ai/docs/pipelines/run-pipeline). For more information about how to create a pipeline run from a pipeline template, see Create, upload, and use a pipeline template (/vertex-ai/docs/pipelines/create-pipeline-template).

For information about capturing and storing pipeline run metadata using Vertex ML Metadata, see Use Vertex ML Metadata to track the lineage of ML artifacts (#track-lineage).

For information about using pipeline runs to experiment on your ML workflow using Vertex AI Experiments, see Add your pipeline runs to experiments (#experiments).

Track the lineage of ML artifacts

A pipeline run contains several artifacts and parameters, including pipeline metadata. To understand changes in the performance or accuracy of your ML system, you need to analyze the metadata and the lineage of ML artifacts from your ML pipeline runs. The lineage of an ML artifact includes all the factors that contributed to its creation, along with metadata and references to artifacts derived from it.

Lineage graphs help you analyze upstream root cause and downstream impact. Each pipeline run produces a lineage graph of parameters and artifacts that are input into the run, materialized within the run, and output from the run. Metadata that composes this lineage graph is stored in Vertex ML Metadata. This metadata can also be synced to Dataplex Universal Catalog.

- **Use Vertex ML Metadata to track pipeline artifact lineage**

  When you run a pipeline using Vertex AI Pipelines, all parameters and artifact metadata consumed and generated by the pipeline are stored in Vertex ML Metadata. Vertex ML Metadata is a managed implementation of the ML Metadata library in TensorFlow, and supports registering and writing custom metadata schemas. When you create a pipeline

run in Vertex AI Pipelines, metadata from the pipeline run is stored in the default metadata store for the project and region where you execute the pipeline.

- **Use Dataplex Universal Catalog to track pipeline artifact lineage**

  Dataplex Universal Catalog (/dataplex/docs/introduction) is a global and cross-project data fabric integrated with multiple systems within Google Cloud, such as Vertex AI, BigQuery, and Cloud Composer. Within Dataplex Universal Catalog, you can search for a pipeline artifact and view its lineage graph. Note that to prevent artifact conflicts, any resource catalogued in Dataplex Universal Catalog is identified with a fully qualified name (FQN) (/dataplex/docs/fully-qualified-names).

  Learn about Dataplex Universal Catalog usage costs. (/dataplex/pricing)

For more information about tracking the lineage of ML artifacts using Vertex ML Metadata and Dataplex Universal Catalog, see Track the lineage of pipeline artifacts (/vertex-ai/docs/pipelines/lineage).

For more information about visualizing, analyzing, and comparing pipeline runs, see Visualize and analyze pipeline results (/vertex-ai/docs/pipelines/visualize-pipeline). For a list of first-party artifact types defined in Google Cloud Pipeline Components, see ML Metadata artifact types (/vertex-ai/docs/pipelines/artifact-types).

Add pipeline runs to experiments

Vertex AI Experiments lets you track and analyze various model architectures, hyperparameters, and training environments to find the best model for your ML use case. After you create an ML pipeline run, you can associate it with an experiment or experiment run. By doing so, you can experiment with different sets of variables, such as hyperparameters, number of training steps, or iterations.

For more information about experimenting with ML workflows using Vertex AI Experiments, see Introduction to Vertex AI Experiments (/vertex-ai/docs/experiments/intro-vertex-ai-experiments).

What's next

- Learn about the interfaces you can use to define and run pipelines using Vertex AI Pipelines (/vertex-ai/docs/pipelines/interfaces).
- Get started by learning how to define a pipeline using the Kubeflow Pipelines SDK (/vertex-ai/docs/pipelines/build-pipeline).
- Learn how to run a pipeline (/vertex-ai/docs/pipelines/run-pipeline).
- Learn about best practices for implementing custom-trained ML models on Vertex AI (https://cloud.google.com/architecture/ml-on-gcp-best-practices#machine-learning-workflow-orchestration)

  .

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-09-30 UTC.